

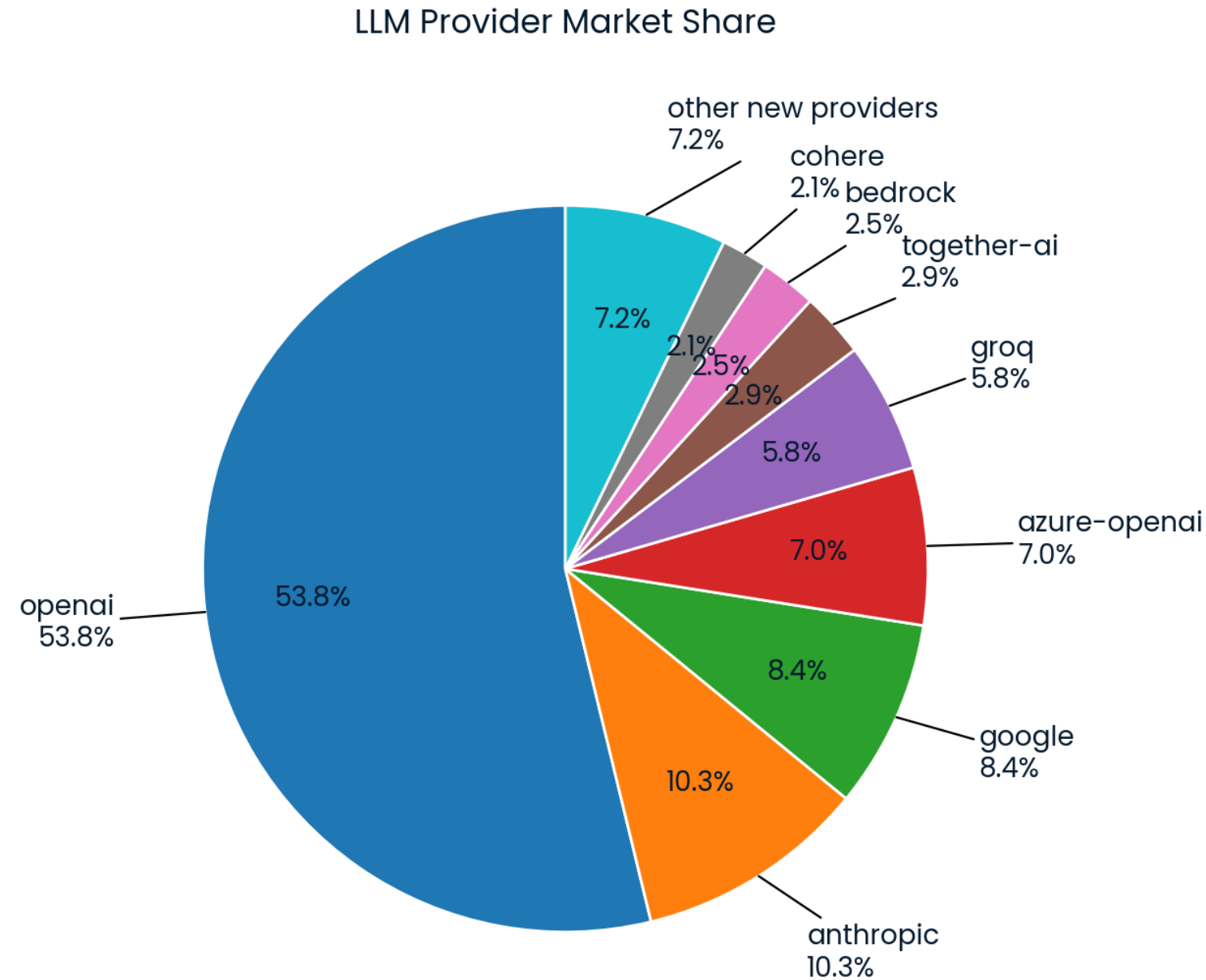
The Responses API: a Developer's Best Friend

WORKING WITH THE OPENAI RESPONSES API



James Chapman
AI Curriculum Manager, DataCamp

OpenAI and the Start of the Generative AI Race



¹ Credit: portkey.ai (as of January, 2025)

OpenAI's API Evolution



OpenAI's API Evolution

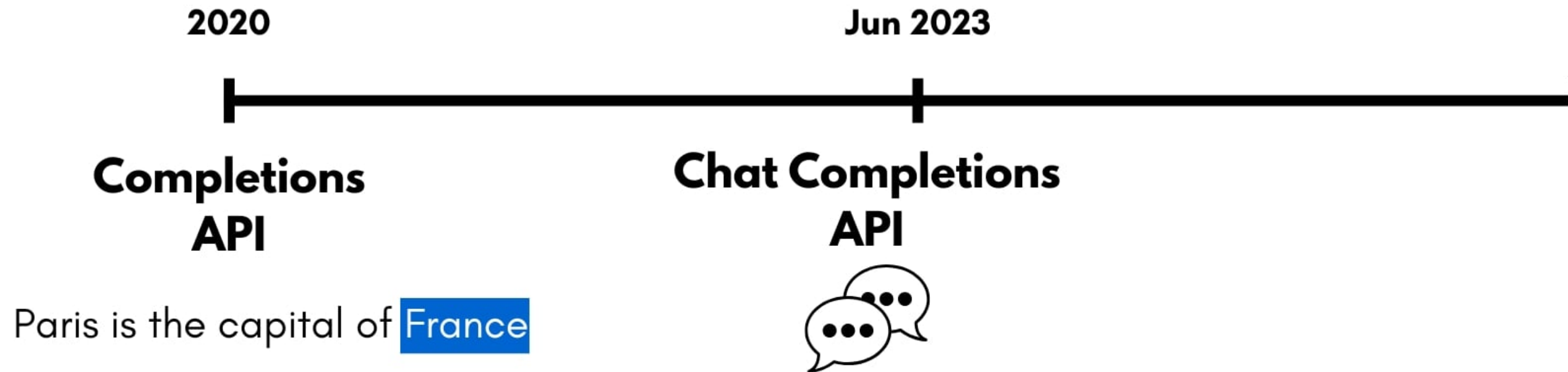
2020



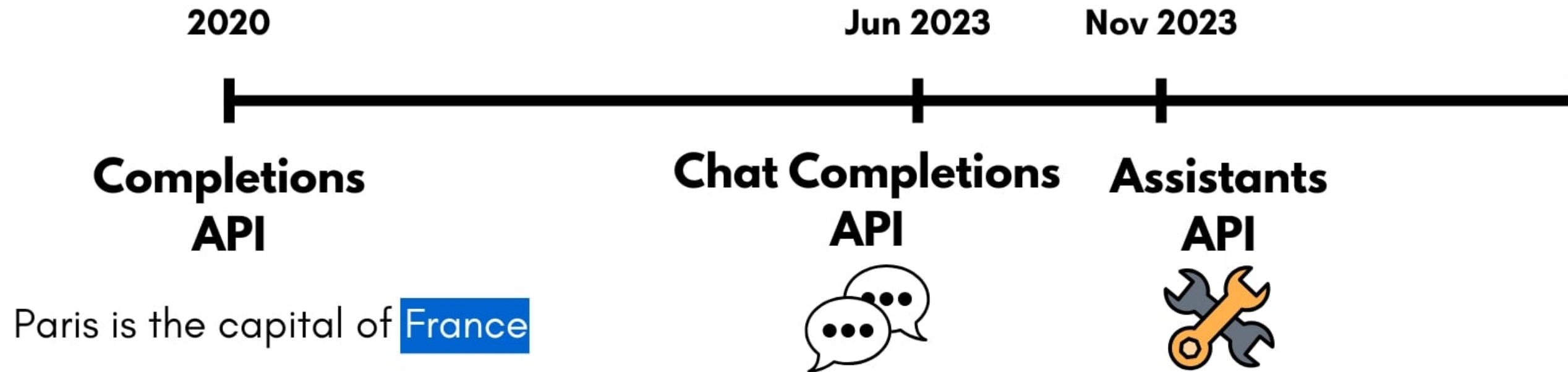
**Completions
API**

Paris is the capital of France

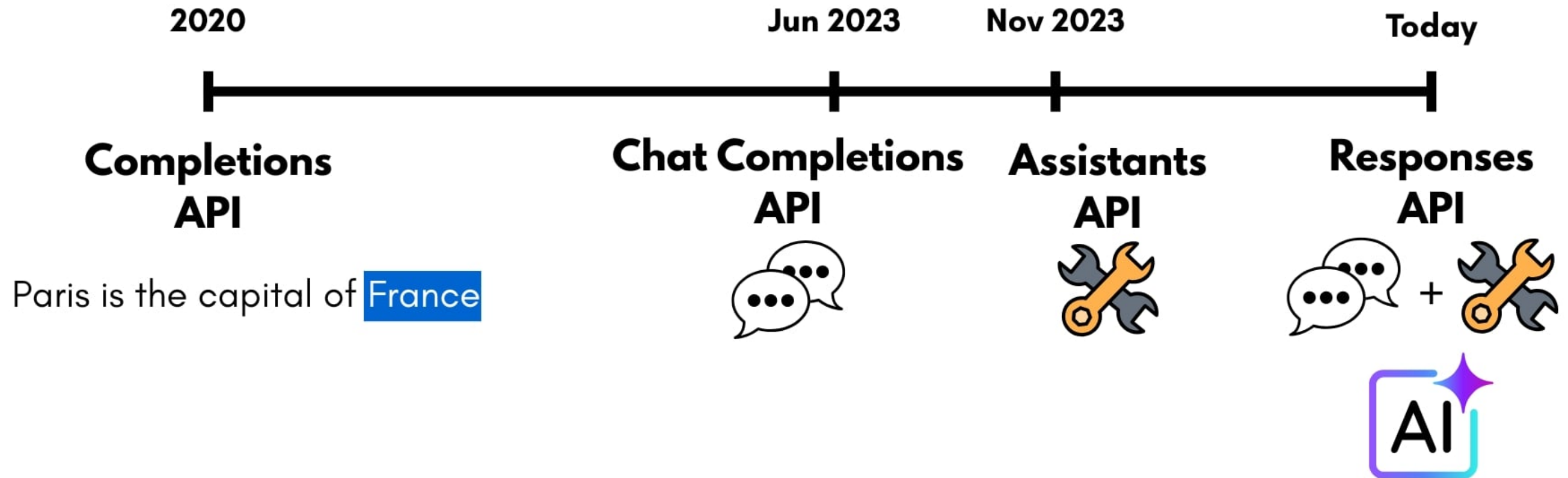
OpenAI's API Evolution



OpenAI's API Evolution



OpenAI's API Evolution



Responses API x Developers

- *Simple, developer-friendly* interface



¹ Image Credit: helicone.ai

Our First Responses API Request

```
from openai import OpenAI

client = OpenAI(api_key="...")

response = client.responses.create(
    model="gpt-5.4-mini",
    reasoning={"effort": "none"},
    max_output_tokens=60,
    instructions="Be concise.",
    input="Explain LLMs to a 6yr old."
)
```

- **Client:** Configure environment to communicate with the API
- **API key is not required** ☐
- Reduce `reasoning` `"effort"` and `max_output_tokens` for faster responses
- `instructions` → clear requirements on how to behave
- `input` → the task or question at-hand
- `instructions` > `input`

```

Response(id='resp_0776f5d708d300430069170352a13081949eb94561de8687c5',
        created_at=1763115858.0,
        error=None,
        incomplete_details=IncompleteDetails(reason='max_output_tokens'),
        instructions='Be concise.',
        metadata={},
        model='gpt-5-mini-2025-08-07',
        object='response',
        output=[ResponseReasoningItem(id='rs_0776f5d708d3004300691703532b788194b83e204f4789d35f', summary=[], type='reasoning', content=None,
encrypted_content=None, status=None), ResponseOutputMessage(id='msg_0776f5d708d3004300691703534dc4819492a95a0516284666', content=
[ResponseOutputText(annotations=[], text='Recursion is a process where something is defined in terms of itself. In computing and mathematics it usually means a function or
procedure that calls (or refers to) itself to solve a problem by breaking it into smaller,', type='output_text', logprobs=[])], role='assistant', status='incomplete', type='message')],
        parallel_tool_calls=True,
        temperature=1.0,
        tool_choice='auto',
        tools=[],
        top_p=1.0,
        background=False,
        conversation=None,
        max_output_tokens=60,
        max_tool_calls=None,
        previous_response_id=None,
        prompt=None,
        prompt_cache_key=None, prompt_cache_retention=None,
        reasoning=Reasoning(effort='minimal', generate_summary=None, summary=None),
        safety_identifier=None,
        service_tier='default',
        status='incomplete',
        text=ResponseTextConfig(format=ResponseFormatText(type='text'), verbosity='medium'),
        top_logprobs=0,
        truncation='disabled',
        usage=ResponseUsage(input_tokens=17, input_tokens_details=InputTokensDetails(cached_tokens=0),
        output_tokens=48,
        output_tokens_details=OutputTokensDetails(reasoning_tokens=0), total_tokens=65),
        user=None,
        billing={'payer': 'developer'},
        store=True)

```

```
print(responses.output_text)
```

```
Imagine a very smart pretend friend...
```

```
print(responses.usage.output_tokens)
```

```
48
```

```
print(responses.id)
```

```
resp_0776f5d708d30043006917035...
```

```
response.output
```

```
[ResponseReasoningItem(
  id='...',
  summary=[],
  type='reasoning',
  content=None,
  encrypted_content=None,
  status=None),
ResponseOutputMessage(
  id='...',
  content=ResponseOutputText(text='Imagi...'
    type='output_text', logprobs=[])],
role='assistant',
status='incomplete',
type='message')]
```

Let's practice!

WORKING WITH THE OPENAI RESPONSES API

Mastering Response Parameters

WORKING WITH THE OPENAI RESPONSES API



James Chapman

AI Curriculum Manager, DataCamp

Model Selection

```
response = client.responses.create(  
    model="gpt-5.4-mini",  
    input="What is recursion?",  
)
```

¹ <https://platform.openai.com/docs/models>

Model Selection

```
response = client.responses.create(  
    model="gpt-5.4-mini",  
    input="What is recursion?",  
)
```

- **Rule-of-thumb:** use the cheapest model that meets the other requirements

Model	Speed	Cost	Best For
gpt-5.4-mini	Fast	Low	Simple tasks, Q&A
gpt-5.5	Moderate	Higher	Complex reasoning

¹ <https://platform.openai.com/docs/models>

LLMs and Tokens

- **Tokens:** units of text that help the AI understand and interpret text

	TOKENS	CHARACTERS
	1	1
H		

¹ <https://platform.openai.com/tokenizer>

LLMs and Tokens

- **Non-reasoning LLMs:** Generates the tokens with the highest probability of following the prompt



LLMs and Tokens



Reasoning Effort

```
response = client.responses.create(  
    model="gpt-5.4-mini",  
    input="Explain LLMs to a 6yr old.",  
    reasoning={"effort": "none"}  
)
```

Reasoning Effort

```
response = client.responses.create(  
    model="gpt-5.4-mini",  
    input="Explain LLMs to a 6yr old.",  
    reasoning={"effort": "none"}  
)
```

Effort	Best For
none	Trivial or mechanical tasks.
low	Simple tasks prioritizing speed and cost.
medium	Default for balanced reasoning and efficiency.
high	Complex, multi-step, or logic-heavy tasks.
xhigh	The hardest tasks that push the boundaries of what's possible

Reasoning Summaries

```
response = client.responses.create(  
    model="gpt-5.4-mini",  
    input="Explain LLMs to a 6yr old.",  
    reasoning={  
        "effort": "medium",  
        "summary": "auto"  
    }  
)
```

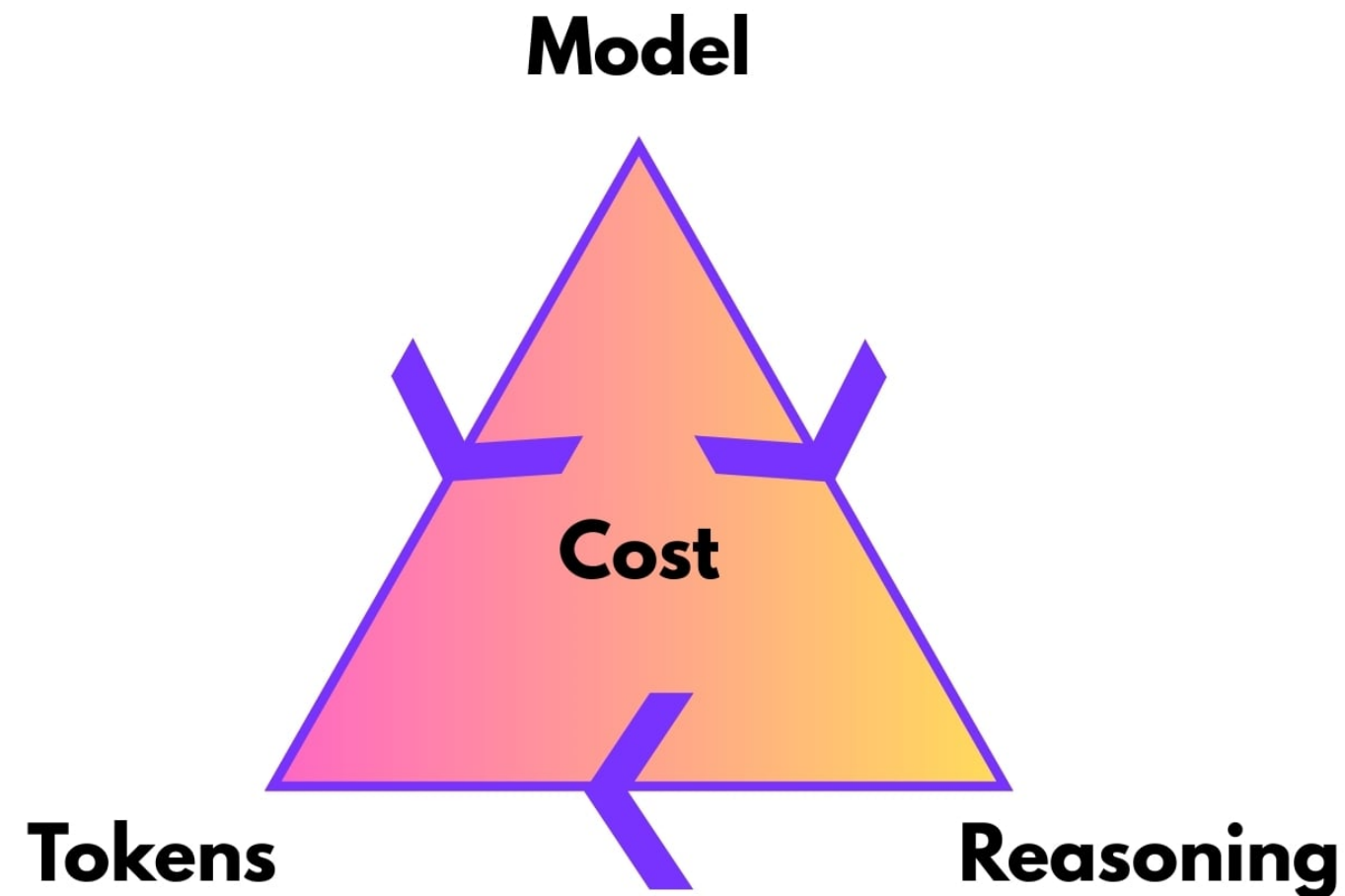
Limiting Output Tokens

```
response = client.responses.create(  
    model="gpt-5.4-mini",  
    input="Explain LLMs to a 6yr old.",  
    reasoning={"effort": "none"},  
    max_output_tokens=500  
)
```

- `max_output_tokens` includes the *reasoning tokens*

Summary

- **Simpler Tasks** → start with small models, no reasoning, and few tokens
- **Complex Tasks** → start with medium model sizes and reasoning, then fine-tune



Let's practice!

WORKING WITH THE OPENAI RESPONSES API

Conversation Chaining

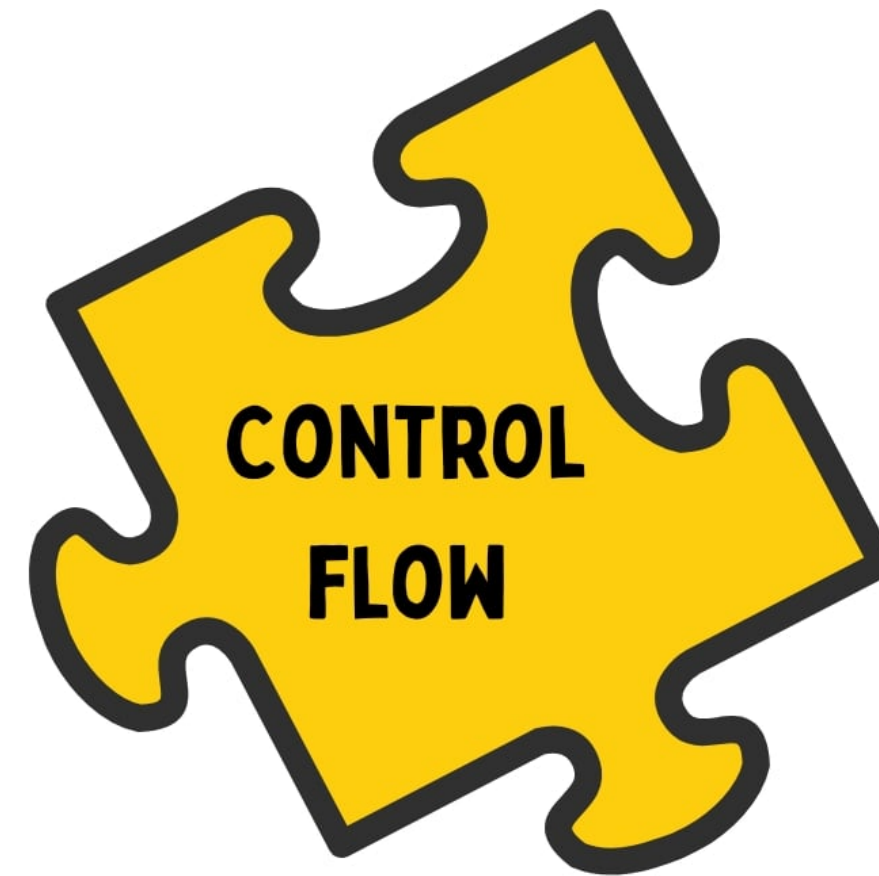
WORKING WITH THE OPENAI RESPONSES API



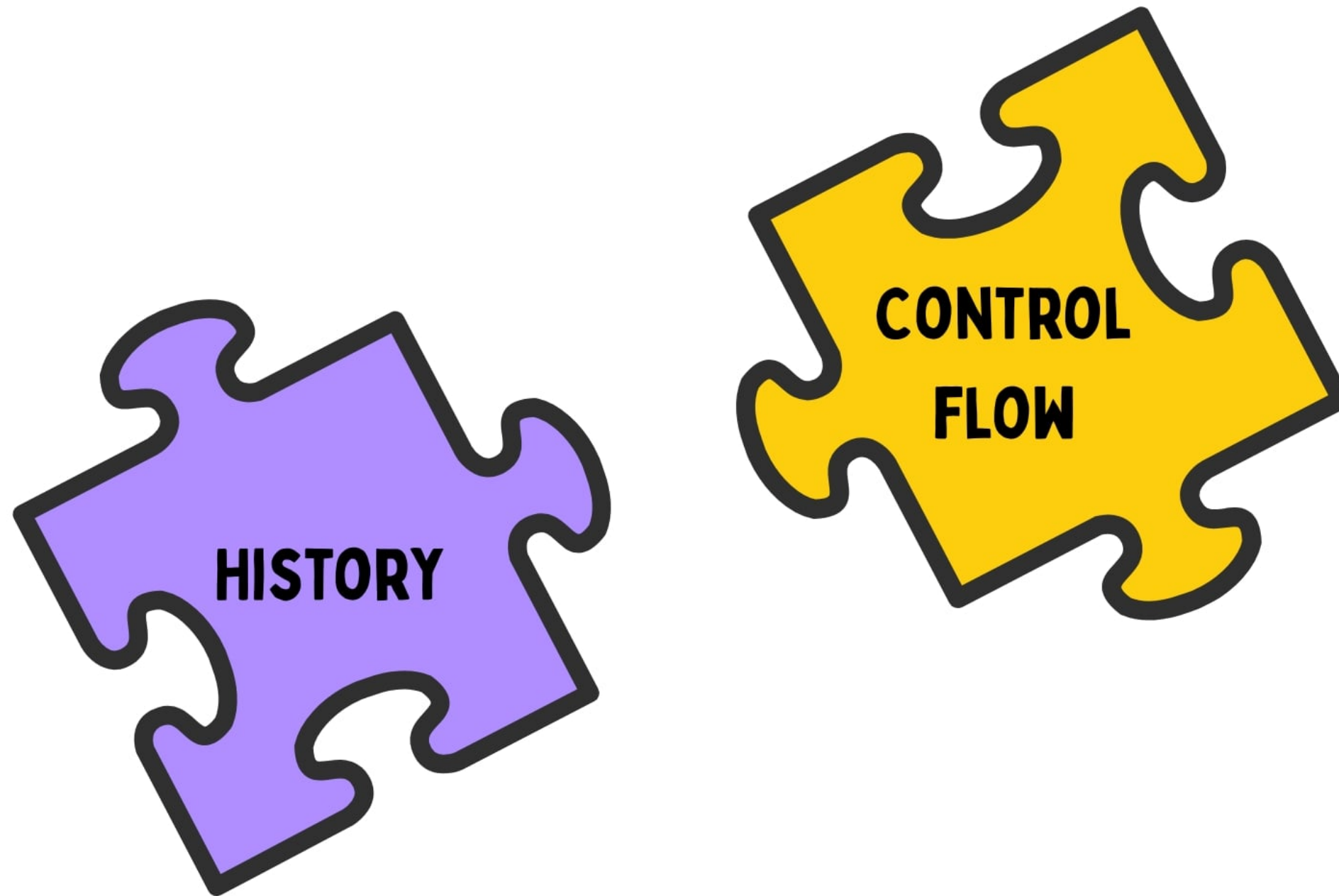
James Chapman

AI Curriculum Manager, DataCamp

Solving the Conversation Puzzle



Solving the Conversation Puzzle



The Conversation History

```
response1 = client.responses.create(  
    model="gpt-5.4-mini",  
    input="Hello, my name is James."  
)  
  
print(response1.id)  
print(response1.output_text)
```

```
resp_02c689cbc16a0b9900691b012ec0f88196a5...  
Hi James - nice to meet you. How can I help?
```

```
response2 = client.responses.create(  
    model="gpt-5.4-mini",  
    input="What is my name?",  
    previous_response_id=response1.id  
)  
  
print(response2.id)  
print(response2.output_text)
```

```
resp_02c689cbc16a0b9900691b01327e4c8196aa...  
Your name is James.
```

The Conversation History

```
response3 = client.responses.create(  
    model="gpt-5.4-mini",  
    input="Summarize the conversation so far.",  
    previous_response_id=response2.id  
)  
  
print(response3.id)  
print(response3.output_text)
```

```
resp_02c689cbc16a0b9900691b0133cf9c81969b1e128919fe3760
```

```
You told me your name is James. I greeted you and later confirmed your name  
when you asked.
```

```
sys_prompt = "You are a helpful Teacher who provides concise, personalized explanations."
latest_response_id = None

while True:
    user_input = input("You: ").strip()
    if user_input.lower() == "exit":
        break

    response = client.responses.create(
        model="gpt-5.4-mini",
        instructions=sys_prompt,
        input=user_input,
        previous_response_id=latest_response_id
    )

    print(f"\nAssistant: {response.output_text}\n")
    latest_response_id = response.id
```





Let's practice!

WORKING WITH THE OPENAI RESPONSES API